

Da rede neural à matriz de rigidez

O que acontece em `plate_with_hole_stiffness.py`, explicado com a teoria

Projeto Struct_Sept, HiWi, TU Wien · 8 de setembro de 2026

Referência: Kofler, Giritsch, Elgeti, *Structural optimization of lattice structures using deep neural networks as geometry representation*, Graphical Models 142 (2025)

Resumo. O script pega uma placa de treliça (*lattice*) com um furo, cuja geometria não é uma malha nem um campo de densidade, mas uma *função*: a distância com sinal até a superfície, calculada por uma rede neural. Ele transforma essa função em triângulos, os triângulos em tetraedros, e os tetraedros na matriz de rigidez global $\mathbf{K}$ de um problema de elasticidade linear. E para aí. Este documento acompanha cada uma dessas etapas com a teoria por trás: o que é uma SDF, como o DeepSDF a representa, por que a treliça precisa de uma função de transformação, o que o FlexiCubes e o tetgen fazem, como a $\mathbf{k}_e$ de um tetraedro sai da forma fraca, e o que a $\mathbf{K}$ resultante tem de ter para estar certa. Todos os números citados foram medidos com o próprio script.

Conteúdo

1	Visão geral: o que entra e o que sai	2
2	A geometria como função: distância com sinal	2
2.1	Definição	2
2.2	Operações booleanas	3
2.3	O furo	3
3	A célula unitária como rede neural: DeepSDF	4
3.1	A ideia	4
3.2	O modelo usado aqui	4
4	Do bloco à placa: empilhar células e graduar o latente	4
4.1	A função de transformação	4
4.2	O campo latente	5
4.3	Fechar as bordas	5
5	Da função à malha	6
5.1	Extração da superfície: FlexiCubes	6
5.2	Deformação livre: do cubo para metros	6
5.3	Preencher o interior: tetgen	7
5.4	Orientação	7
6	Elementos finitos: da elasticidade à matriz de rigidez	8
6.1	O problema contínuo	8
6.2	A forma fraca	8
6.3	Discretização com tetraedros lineares	8
6.4	Montagem	9
6.5	Esparsidade	9
7	O que a matriz de rigidez tem de satisfazer	10
7.1	Simetria	10
7.2	Semidefinida positiva, com núcleo de dimensão seis	10
7.3	O que os números da corrida dizem	11
8	O que falta, e por que o script para aqui	11
9	Resumo em uma tabela	12

1 Visão geral: o que entra e o que sai

O pipeline do artigo tem seis etapas. As duas primeiras (amostrar distâncias e treinar a rede) já foram feitas uma vez e os pesos estão salvos. O script executa as etapas 3 a 5, cortando a 5 no ponto em que a matriz de rigidez existe:

Etapa	O que acontece	Função do script
3	A rede e a treliça viram uma única função $\varphi(\xi)$ no cubo $[0, 1]^3$; o furo é subtraído; as bordas são fechadas.	plate_with_hole
4a	FlexiCubes extrai a superfície $\varphi = 0$ como triângulos e o spline de deformação os leva para metros.	tetrahedral_mesh
4b	tetgen preenche a superfície fechada com tetraedros.	tetrahedral_mesh
5a	Cada tetraedro ganha uma matriz $\mathbf{k}_e$ (12×12); todas são somadas em $\mathbf{K}$.	build_solid, assemble_stiffness
5b	<i>Não feito</i> : condições de contorno, cargas, solução de $\mathbf{K}\mathbf{u} = \mathbf{f}$.	—
6	<i>Não feito</i> : sensibilidades e atualização das variáveis de projeto (MMA).	—

Tabela 1: As etapas do pipeline e onde o script para.

Um ponto que organiza tudo o que vem a seguir: **há dois sistemas de coordenadas**. A rede e a treliça vivem no cubo paramétrico $\xi \in [0, 1]^3$. A placa física, de $1 \times 1 \times 0,1$ m, só aparece no fim, quando um spline leva os vértices da malha para metros. Sempre que uma quantidade é definida em metros (o furo, o material, a $\mathbf{K}$) e outra no cubo (a rede, a treliça), alguma conversão tem de acontecer no meio, e o script tem uma classe só para isso.

2 A geometria como função: distância com sinal

2.1 Definição

Seja $\Omega \subset \mathbb{R}^3$ um sólido e $\Gamma = \partial\Omega$ a sua superfície. A **função distância com sinal** (SDF, *signed distance function*) é

$$\varphi(\mathbf{x}) = \begin{cases} -\min_{\mathbf{y} \in \Gamma} \|\mathbf{x} - \mathbf{y}\| & \text{se } \mathbf{x} \in \Omega \text{ (dentro)} \\ +\min_{\mathbf{y} \in \Gamma} \|\mathbf{x} - \mathbf{y}\| & \text{caso contrário (fora)} \end{cases} \quad (1)$$

A superfície é o conjunto de nível zero, $\Gamma = \{\mathbf{x} : \varphi(\mathbf{x}) = 0\}$, e o material é $\Omega = \{\varphi < 0\}$. Uma SDF exata satisfaz $\|\nabla\varphi\| = 1$ em quase todo ponto: andar uma unidade em direção à superfície reduz a distância em exatamente uma unidade (Figura 1).

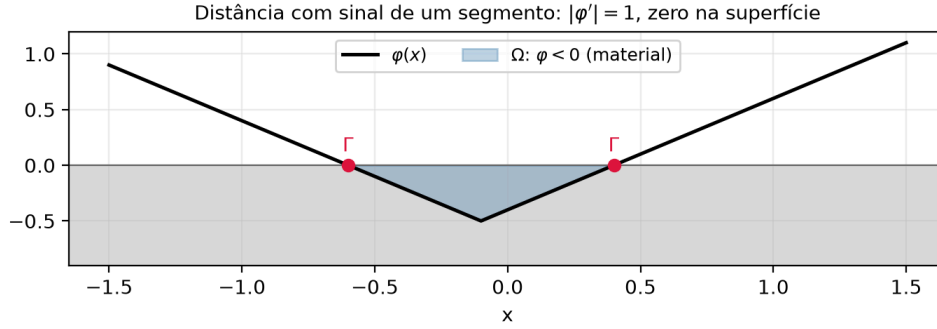


Figura 1: Uma SDF em uma dimensão. O sólido é o segmento entre os dois pontos vermelhos; dentro dele $\varphi < 0$. A inclinação é ± 1 em todo lugar.

Por que representar geometria assim, e não por uma malha? Porque uma função pode ser avaliada em qualquer ponto, combinada com outras funções e, sobretudo, **derivada**: se φ depende de parâmetros, a superfície $\varphi = 0$ também depende, de forma contínua. É isso que o otimizador precisa.

2.2 Operações booleanas

Duas SDFs se combinam com $\min$ e $\max$:

$$\varphi_{A \cup B} = \min(\varphi_A, \varphi_B), \quad \varphi_{A \cap B} = \max(\varphi_A, \varphi_B), \quad \varphi_{A \setminus B} = \max(\varphi_A, -\varphi_B). \quad (2)$$

A intuição para a diferença $A \setminus B$: negar φ_B troca dentro por fora, e a interseção com “o fora de B ” é exatamente “ A menos B ”. Estas fórmulas dão o **conjunto de nível zero exato**, mas o valor longe da superfície é apenas um limite inferior da distância verdadeira (perto de cantos côncavos elas subestimam). Para extrair a superfície isso não importa.

2.3 O furo

Um cilindro de raio r com eixo z passando por (c_x, c_y) tem, no plano,

$$\varphi_{\text{cil}}(\mathbf{x}) = \sqrt{(x - c_x)^2 + (y - c_y)^2} - r. \quad (3)$$

O `CylinderSDF` da biblioteca é a versão 3D com tampas; o script o faz atravessar a placa inteira (de $z = -t$ a $z = 2t$) para que a tampa nunca caia dentro do material.

No código `DifferenceSDF(lattice, ScaledSpaceSDF(hole, [L, W, t]))` implementa Equação 2. A `ScaledSpaceSDF` existe por causa dos dois sistemas de coordenadas: o furo é descrito em metros, a treliça no cubo. A classe recebe o ponto paramétrico ξ , estica-o para metros, $\mathbf{x} = \mathbf{S}\xi$ com $\mathbf{S} = \text{diag}(L, W, t)$, avalia o furo lá e devolve

$$\tilde{\varphi}_{\text{furo}}(\xi) = \frac{\varphi_{\text{furo}}(\mathbf{S}\xi)}{\bar{S}}, \quad (4)$$

com $\bar{S}$ a média dos três fatores. O zero é exato; as magnitudes são apenas aproximadas quando a placa não é cúbica, porque um escalar não desfaz um estiramento anisotrópico. O `FlexiCubes` só usa as magnitudes para posicionar vértices, e ordem de grandeza certa basta.

3 A célula unitária como rede neural: DeepSDF

3.1 A ideia

Uma rede DeepSDF é uma função $f_\theta(\mathbf{x}, \boldsymbol{\lambda}) \approx \varphi_\lambda(\mathbf{x})$ que, dado um ponto $\mathbf{x} \in [-1, 1]^3$ e um **vetor latente** $\boldsymbol{\lambda}$, devolve a distância com sinal até a superfície da geometria codificada por $\boldsymbol{\lambda}$. Os pesos θ são treinados uma vez sobre uma família de formas; depois disso, mudar $\boldsymbol{\lambda}$ percorre continuamente essa família. É um espaço de projeto de baixa dimensão que só contém geometrias “parecidas com as de treino”, ao contrário da otimização topológica por densidades, que pode produzir qualquer coisa.

O treino minimiza, sobre pontos amostrados $\mathbf{x}_i$ e formas j ,

$$\min_{\theta, \{\boldsymbol{\lambda}_j\}} \sum_j \sum_i |f_\theta(\mathbf{x}_i, \boldsymbol{\lambda}_j) - \varphi_j(\mathbf{x}_i)|. \quad (5)$$

Cada forma de treino ganha o seu $\boldsymbol{\lambda}_j$ (inicializado de $\mathcal{N}(0, 0,01)$), e os pesos são compartilhados.

3.2 O modelo usado aqui

O script carrega `PretrainedModels.AnalyticRoundCross`, o caso de teste 1 do artigo: três cilindros perpendiculares de raio r , com **latente de dimensão 1** e $\boldsymbol{\lambda} = r$. Apesar do nome “decoder”, este modelo em particular não passa pelos pesos: o **forward** calcula a fórmula fechada

$$f(\mathbf{x}, r) = \min\left(\|\mathbf{x}\|_\infty, \sqrt{y^2 + z^2} - r, \sqrt{x^2 + z^2} - r, \sqrt{x^2 + y^2} - r\right), \quad (6)$$

que é a união Equação 2 de três cilindros. O primeiro termo, $\|\mathbf{x}\|_\infty$, é só o valor inicial do `min`: positivo em todo ponto exceto a origem, não adiciona material, e garante que longe dos cilindros a distância cresça. É a geometria de referência com erro de reconstrução zero; os modelos `RoundCross`, `ChiAndCross` e `Primitives*` são redes de verdade com a mesma interface.¹

Atenção O latente só faz sentido na faixa em que a rede foi treinada, $r \in [0,15, 0,75]$. Fora dela, uma rede de verdade devolve geometria sem significado. O script usa $r = 0,4$ constante.

4 Do bloco à placa: empilhar células e graduar o latente

4.1 A função de transformação

A rede conhece uma célula no cubo $[-1, 1]^3$. A placa tem $t_x \times t_y \times t_z$ células, aqui $4 \times 4 \times 1$. A treliça é obtida com uma única função T , aplicada coordenada a coordenada, que mapeia o cubo paramétrico $[0, 1]$ na entrada da rede $[-1, 1]$ (eq. 18 do artigo):

$$T(x) = 4 \left\lfloor t_x \frac{x}{2} - \left\lfloor t_x \frac{x}{2} + \frac{1}{2} \right\rfloor \right\rfloor - 1, \quad \varphi_{\text{treliça}}(\boldsymbol{\xi}) = f_\theta(T(\xi_1), T(\xi_2), T(\xi_3), \boldsymbol{\lambda}(\boldsymbol{\xi})). \quad (7)$$

¹Verificado destruindo os pesos com ruído: a saída não muda. Vale a pena trocar para `RoundCross` e comparar as duas lado a lado, é o item 7 da fila em `playground/IDEIAS.md`.

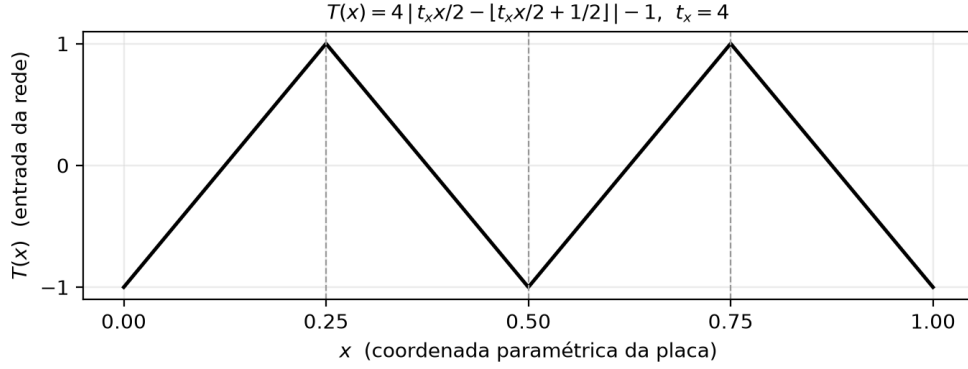


Figura 2: A função de transformação para $t_x = 4$. Cada célula vai de -1 a 1 ; a seguinte volta de 1 a -1 . As linhas tracejadas são as interfaces entre células.

T é uma onda triangular (Figura 2): dentro de uma célula ela é uma reta, e na interface entre células ela **espelha** em vez de saltar. É por isso que a SDF resultante é contínua mesmo para células assimétricas, que é o caso normal de geometria representada por rede: uma célula e a sua vizinha são imagens espelhadas uma da outra, então coincidem na interface. A alternativa da literatura (Chabra et al., *Deep Local Shapes*) usa uma transformação descontínua por célula mais uma função de peso; o artigo evita isso deliberadamente.

No código `LatticeSDFStruct(tiling=[4, 4, 1], microtile=SDFfromDeepSDF(model), parametrization=Constant([0.4]))` implementa Equação 7. A função `transform` em `lattice_structure.py` é literalmente `2 * abs(t*x/2 - floor((t*x + 1)/2))`, reescalada para $[-1, 1]$.

4.2 O campo latente

No artigo o latente varia no espaço, $\lambda(\xi)$, e é isso que *gradua* a treliça: barras grossas onde o esforço é grande, finas onde não é. A variação é dada por um B-spline com suporte local (eq. 21 do artigo),

$$\lambda(\xi) = \sum_i N_i(\xi) \hat{\lambda}_i, \quad (8)$$

e os pontos de controle $\hat{\lambda}_i$ são as **variáveis de projeto** da otimização. Suporte local significa que mover um ponto de controle muda a treliça só numa vizinhança, sem perturbar o resto. O número de pontos de controle é independente do número de células.

Este script usa `Constant([0.4])`: $\lambda(\xi) = 0,4$ em toda a placa, todas as células iguais. É o caso degenerado de Equação 8 com um único ponto de controle, e é a única coisa que precisa mudar (trocar `Constant` por `SplineParametrization`) para o problema virar um problema de otimização.

4.3 Fechar as bordas

Cortar a treliça pela fronteira do cubo deixa barras abertas: a superfície $\varphi = 0$ não é fechada, e uma superfície aberta não tem interior, logo não tem volume nem tetraedros. `CappedBorderSDF` faz a interseção da treliça com o próprio cubo, $\varphi = \max(\varphi_{\text{treliça}}, \varphi_{\text{cubo}})$ (Equação 2), o que fecha cada barra com uma tampa plana na fronteira. A superfície resultante é estanque (*watertight*) e o script verifica isso antes de chamar o tetgen.

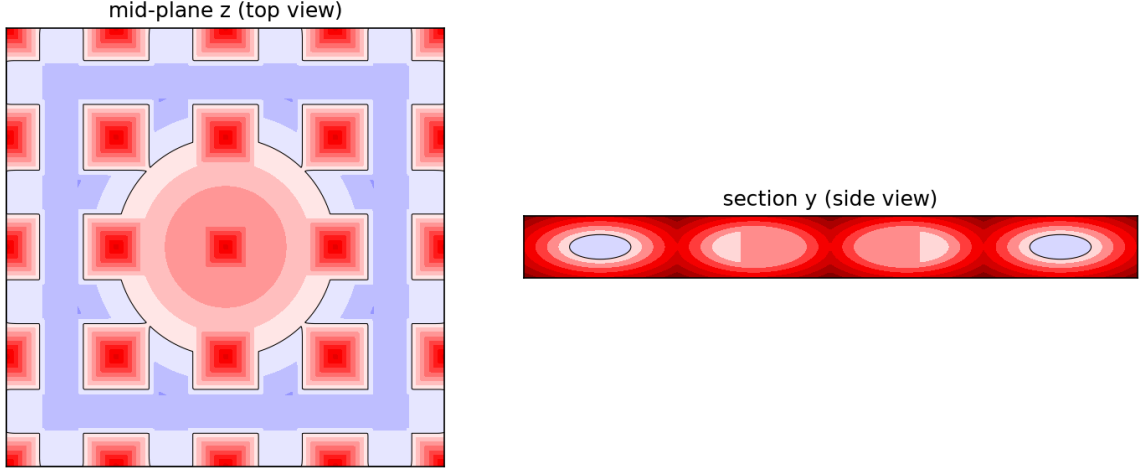


Figura 3: Cortes da SDF final, já em metros. Esquerda: plano médio $z = 0,05$ m visto de cima; direita: seção $y = 0,5$ m. Azul é $\varphi < 0$ (material), vermelho $\varphi > 0$ (vazio); a linha preta é $\varphi = 0$. Vê-se o furo de raio $0,25$ m cortando as células centrais e as tampas retas na borda.

5 Da função à malha

5.1 Extração da superfície: FlexiCubes

O FEM precisa de uma malha. A superfície $\varphi = 0$ é extraída numa grade regular: N pontos por célula e por eixo ($N = 10$, ou seja $41 \times 41 \times 11$ pontos no total), estendida 5 % para fora do cubo para que a superfície feche nas bordas. Em cada cubo da grade cujos vértices têm sinais diferentes de φ , a superfície passa por dentro.

O *marching cubes* clássico coloca um vértice de malha em cada aresta com troca de sinal, por interpolação linear:

$$\mathbf{x}_e = \frac{\varphi_b \mathbf{x}_a - \varphi_a \mathbf{x}_b}{\varphi_b - \varphi_a}, \quad (9)$$

o ponto da aresta (a, b) onde a reta que liga $(\mathbf{x}_a, \varphi_a)$ a $(\mathbf{x}_b, \varphi_b)$ cruza zero. O **FlexiCubes** (Shen et al. 2023) é a versão *dual* e diferenciável: cada cubo cortado recebe um único vértice, colocado numa média ponderada dos cruzamentos Equação 9, e os vértices de cubos vizinhos são ligados por quadriláteros que depois viram triângulos. Os pesos são parâmetros extras que podem ser otimizados junto; aqui não são (`differentiate=False`).

Dois motivos para o FlexiCubes em vez do marching cubes: os triângulos saem bem formados (o marching cubes produz lascas, que são péssimas para o FEM) e, como Equação 9 é uma função diferenciável de φ_a, φ_b , **cada vértice da malha é uma função diferenciável dos valores da SDF**, logo do latente. Essa é a corrente que a otimização precisa manter intacta.

5.2 Deformação livre: do cubo para metros

Os vértices extraídos estão em $[0, 1]^3$. A biblioteca os leva para o domínio físico com um B-spline trilinear (*free-form deformation*),

$$\bar{\mathbf{x}} = \sum_i N_i(\boldsymbol{\xi}) \mathbf{P}_i, \quad (10)$$

que para uma caixa se reduz a $\bar{x} = (L\xi_1, W\xi_2, t\xi_3)$. Aqui é só um estiramento anisotrópico, mas como é um **TorchSpline**, os pontos de controle P_i podem depois entortar a placa em qualquer forma sem tocar no resto do código. No artigo eles **não** são variáveis de projeto.

5.3 Preencher o interior: tetgen

A superfície fechada (8192 triângulos, 4096 vértices) vai para o tetgen, que constrói uma tetraedrização de Delaunay restrita: os triângulos da superfície são preservados como faces (opção Υ) e o interior é preenchido inserindo **pontos de Steiner** onde for preciso para respeitar um critério de qualidade (opção q , que limita a razão raio-aresta). Saem 4667 nós (os 4096 da superfície mais 571 internos) e 16 015 tetraedros. O volume total dos tetraedros, $\sum_e V_e = 0,020241 \text{ m}^3$, coincide com o volume fechado pela superfície em todos os dígitos impressos, que é o teste de que a tetraedrização não perdeu nem inventou material.

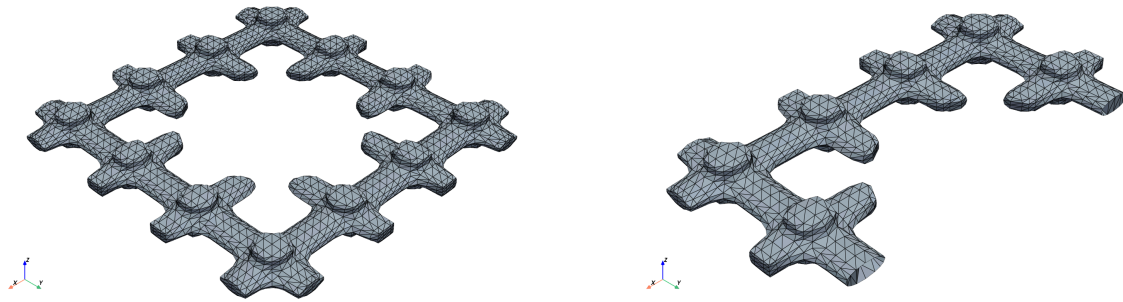


Figura 4: A malha de tetraedros que entra no torch-fem, inteira (esquerda) e cortada pelo plano $y = 0,5 \text{ m}$ (direita). O furo de raio $0,25 \text{ m}$ remove as quatro células centrais; sobra um anel de doze.

Atenção A biblioteca também oferece `create_3D_mesh(mesh_type="volume")`, que usa a tetraedrização interna do FlexiCubes e é o que o teste de otimização da biblioteca usa. Medido num cubo unitário maciço, esse caminho devolve tetraedros que somam apenas 0,68 a 0,70 do volume, em qualquer resolução, e o contorno da malha tem 13 034 triângulos em 5836 componentes (a superfície tem 1200 em uma). São milhares de cavidades internas. A matriz de rigidez de uma malha assim é a de um corpo com 30 % de vazios. Por isso o script usa o tetgen, cujo volume bate com o da superfície.

5.4 Orientação

Antes de virar elemento finito, cada tetraedro precisa de orientação positiva. O volume com sinal

$$V_e = \frac{1}{6}(\mathbf{x}_1 - \mathbf{x}_0) \cdot [(\mathbf{x}_2 - \mathbf{x}_0) \times (\mathbf{x}_3 - \mathbf{x}_0)] \quad (11)$$

é positivo se os quatro nós seguem a regra da mão direita. Se for negativo, trocar dois nós inverte o sinal sem mudar a geometria. O motivo de isso importar aparece em Equação 18: o determinante do Jacobiano do elemento é $6V_e$, e um determinante negativo daria uma $\mathbf{k}_e$ negativa. Tetraedros de volume nulo (quatro nós coplanares) têm Jacobiano singular e são descartados. Com a saída do tetgen os dois passos não fazem nada, mas custam nada e protegem contra surpresas.

6 Elementos finitos: da elasticidade à matriz de rigidez

6.1 O problema contínuo

Um sólido elástico linear ocupando $\bar{\Omega}$ obedece, em cada ponto, ao equilíbrio, à cinemática de pequenas deformações e à lei de Hooke:

$$\nabla \cdot \boldsymbol{\sigma} + \mathbf{b} = \mathbf{0}, \quad \boldsymbol{\varepsilon} = \frac{1}{2}(\nabla \mathbf{u} + \nabla \mathbf{u}^T), \quad \boldsymbol{\sigma} = \mathbf{C} : \boldsymbol{\varepsilon}. \quad (12)$$

Para material isotrópico, $\boldsymbol{\sigma} = \lambda_L \operatorname{tr}(\boldsymbol{\varepsilon})\mathbf{I} + 2\mu\boldsymbol{\varepsilon}$, com as constantes de Lamé

$$\lambda_L = \frac{E\nu}{(1+\nu)(1-2\nu)}, \quad \mu = \frac{E}{2(1+\nu)}. \quad (13)$$

Com aço, $E = 210$ GPa e $\nu = 0,3$, vem $\lambda_L \approx 121$ GPa e $\mu \approx 81$ GPa. Em notação de Voigt, $\boldsymbol{\varepsilon} = (\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{zz}, \gamma_{xy}, \gamma_{yz}, \gamma_{xz})^T$ e $\mathbf{C}$ é a matriz 6×6

$$\mathbf{C} = \begin{bmatrix} \lambda_L + 2\mu & \lambda_L & \lambda_L & 0 & 0 & 0 \\ \lambda_L & \lambda_L + 2\mu & \lambda_L & 0 & 0 & 0 \\ \lambda_L & \lambda_L & \lambda_L + 2\mu & 0 & 0 & 0 \\ 0 & 0 & 0 & \mu & 0 & 0 \\ 0 & 0 & 0 & 0 & \mu & 0 \\ 0 & 0 & 0 & 0 & 0 & \mu \end{bmatrix}. \quad (14)$$

No código `torchfem.materials.IsotropicElasticity3D(E=210e9, nu=0.3)` é Equação 14. A biblioteca a “vetoriza”: uma cópia por elemento, o que permitiria material diferente em cada tetraedro.

6.2 A forma fraca

Multiplicando o equilíbrio por um deslocamento virtual $\mathbf{v}$ (nulo onde $\mathbf{u}$ é prescrito) e integrando por partes, chega-se ao princípio dos trabalhos virtuais: encontrar $\mathbf{u}$ tal que, para todo $\mathbf{v}$ admissível,

$$\underbrace{\int_{\Omega} \boldsymbol{\varepsilon}(\mathbf{v}) : \mathbf{C} : \boldsymbol{\varepsilon}(\mathbf{u}) \, d\Omega}_{a(\mathbf{u}, \mathbf{v}) \text{ -- trabalho interno}} = \underbrace{\int_{\Omega} \mathbf{v} \cdot \mathbf{b} \, d\Omega + \int_{\Gamma_N} \mathbf{v} \cdot \mathbf{t} \, d\Gamma}_{\ell(\mathbf{v}) \text{ -- trabalho externo}}. \quad (15)$$

A forma bilinear $a(\cdot, \cdot)$ é simétrica (porque $\mathbf{C}$ é) e é ela que vira a matriz de rigidez. O lado direito vira o vetor de forças. A rigidez, portanto, **não depende das cargas nem dos apoios**: só de geometria e material. É por isso que o script pode montá-la sem ter decidido nada sobre condições de contorno.

6.3 Discretização com tetraedros lineares

Dentro de um tetraedro de nós $\mathbf{x}_1 \dots \mathbf{x}_4$, o deslocamento é interpolado linearmente,

$$\mathbf{u}(\mathbf{x}) = \sum_{i=1}^4 N_i(\mathbf{x}) \mathbf{u}_i, \quad (16)$$

com N_i as **coordenadas baricêntricas**: $N_i = 1$ no nó i , 0 nos outros, lineares em $\mathbf{x}$ e somando 1. Cada $N_i = (a_i + b_i x + c_i y + d_i z)/(6V_e)$, de modo que $\nabla N_i = (b_i, c_i, d_i)/(6V_e)$ é **constante** no elemento. A deformação de Voigt fica $\boldsymbol{\varepsilon} = \mathbf{B} \mathbf{u}_e$, com $\mathbf{u}_e \in \mathbb{R}^{12}$ os 4 nós $\times$ 3 componentes e

$$\mathbf{B} = [\mathbf{B}_1 \ \mathbf{B}_2 \ \mathbf{B}_3 \ \mathbf{B}_4], \quad \mathbf{B}_i = \frac{1}{6V_e} \begin{bmatrix} b_i & 0 & 0 \\ 0 & c_i & 0 \\ 0 & 0 & d_i \\ c_i & b_i & 0 \\ 0 & d_i & c_i \\ d_i & 0 & b_i \end{bmatrix}. \quad (17)$$

Substituindo em $a(\mathbf{u}, \mathbf{v})$ restrito a um elemento, como $\mathbf{B}$ e $\mathbf{C}$ são constantes, a integral é trivial:

$$\mathbf{k}_e = \int_{\Omega_e} \mathbf{B}^T \mathbf{C} \mathbf{B} d\Omega = V_e \mathbf{B}^T \mathbf{C} \mathbf{B} \quad (12 \times 12). \quad (18)$$

Um único ponto de integração é **exato** para este elemento. Em unidades: $\mathbf{C}$ em Pa, $\mathbf{B}$ em 1/m, V_e em m^3 , logo $\mathbf{k}_e$ em N/m, rigidez de mola.

No código `solid.k0()` calcula Equação 18 para os 16 015 elementos de uma vez, tensor de forma (16015, 12, 12). O `torch-fem` faz isso pelo caminho isoparamétrico: mapeia um tetraedro de referência para o físico, $\mathbf{J} = \partial \mathbf{x} / \partial \boldsymbol{\xi}$, obtém $\nabla N_i = \mathbf{J}^{-T} \nabla_{\boldsymbol{\xi}} N_i$ e integra $w \cdot \det \mathbf{J} \cdot \mathbf{B}^T \mathbf{C} \mathbf{B}$ com $w = 1/6$ e $\det \mathbf{J} = 6V_e$, o que dá exatamente $V_e \mathbf{B}^T \mathbf{C} \mathbf{B}$. Este é o lugar onde um tetraedro invertido faria estrago: $\det \mathbf{J} < 0$ produziria $\mathbf{k}_e$ negativa.

6.4 Montagem

Os deslocamentos nodais do elemento são um recorte do vetor global: $\mathbf{u}_e = \mathbf{L}_e \mathbf{u}$, com $\mathbf{L}_e$ uma matriz booleana $12 \times n$ que seleciona os graus de liberdade (GDL) $3i + d$ dos quatro nós, $d \in \{0, 1, 2\}$ para x, y, z . A energia total é a soma das energias dos elementos, e daí

$$\mathbf{K} = \sum_{e=1}^{n_e} \mathbf{L}_e^T \mathbf{k}_e \mathbf{L}_e \quad (n \times n), \quad n = 3n_{\text{nós}} = 14001. \quad (19)$$

Ninguém forma as $\mathbf{L}_e$ de verdade: cada entrada (i, j) de $\mathbf{k}_e$ é **somada** na posição $(\text{gdl}_i, \text{gdl}_j)$ de $\mathbf{K}$ (*scatter-add*). Como nós compartilhados por vários elementos recebem contribuições de todos eles, a mesma posição é atingida várias vezes, e é a soma que faz a estrutura se comportar como um todo em vez de como 16 015 tetraedros soltos.

No código `solid.assemble_matrix(k, con)` implementa Equação 19 em formato esparsa COO: gera as triplas (linha, coluna, valor) de todos os elementos, $16\,015 \times 144 = 2,3$ milhões delas, e `coalesce()` soma as que caem na mesma posição, sobrando 488 007 entradas distintas. O argumento `con` lista GDL prescritos, cujas linhas e colunas são trocadas pela identidade; o script o passa vazio de propósito.

6.5 Esparsidade

$K_{ij} \neq 0$ só se os GDL i e j pertencem a algum elemento em comum, ou seja, se os nós são vizinhos na malha. $\mathbf{K}$ é, portanto, a matriz de adjacência da malha com blocos 3×3 nas entradas. Na placa, cada linha tem em média 35 não-zeros (o nó e os seus cerca de 11 vizinhos, vezes 3), e a matriz tem 0,25 % de preenchimento. A Figura 5 mostra o padrão: a banda diagonal são os 4096 nós de superfície, numerados pelo FlexiCubes ao longo da grade (por isso vizinhos no espaço são vizinhos no índice); o bloco denso no canto são os 571 nós internos que o tetgen inseriu depois, sem ordem espacial.

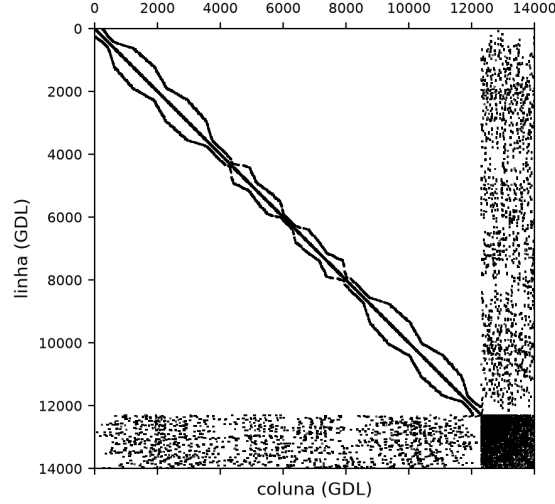


Figura 5: Padrão de esparsidade de $\mathbf{K}$ (14001×14001 , 488 007 não-zeros). Cada ponto preto é uma entrada não nula.

7 O que a matriz de rigidez tem de satisfazer

Três propriedades seguem da teoria, e o script mede as três. Elas servem como testes porque cada uma falha de um jeito diferente.

7.1 Simetria

$\mathbf{K}^T = \mathbf{K}$ porque $a(\mathbf{u}, \mathbf{v}) = a(\mathbf{v}, \mathbf{u})$, ou, algebricamente, porque $(\mathbf{B}^T \mathbf{C} \mathbf{B})^T = \mathbf{B}^T \mathbf{C}^T \mathbf{B}$ e $\mathbf{C}$ é simétrica. Medido: $\max |K_{ij} - K_{ji}| / \max |K_{ij}| = 2 \times 10^{-16}$, o epsilon da máquina em precisão dupla. Uma assimetria maior indicaria erro na montagem.

7.2 Semidefinida positiva, com núcleo de dimensão seis

A energia de deformação $\frac{1}{2} \mathbf{u}^T \mathbf{K} \mathbf{u} = \frac{1}{2} \int \boldsymbol{\varepsilon} : \mathbf{C} : \boldsymbol{\varepsilon} d\Omega \geq 0$ para todo $\mathbf{u}$, logo $\mathbf{K}$ é semidefinida positiva e todos os elementos da diagonal são positivos (medido: mínimo $1,1 \times 10^9$ N/m). A igualdade a zero ocorre exatamente quando $\boldsymbol{\varepsilon} = 0$ em todo lugar, ou seja, para deslocamentos que **não deformam**: os movimentos de corpo rígido. Em 3D são seis:

$$\mathbf{u} = \mathbf{c} \quad (\text{três translações}), \quad \mathbf{u} = \boldsymbol{\omega} \times \mathbf{x} \quad (\text{três rotações infinitesimais}). \quad (20)$$

A rotação não deforma porque $\nabla(\boldsymbol{\omega} \times \mathbf{x}) = [\boldsymbol{\omega}]_{\times}$ é antissimétrica, e a parte simétrica de uma matriz antissimétrica é zero. Assim, sem apoios, $\mathbf{K}$ é **singular** com núcleo de dimensão seis: $\mathbf{K} \mathbf{u}_{\text{rígido}} = \mathbf{0}$.

O script constrói os seis vetores de Equação 20 para todos os nós e mede $\max |\mathbf{K} \mathbf{u}_{\text{rígido}}| / \max |K_{ij}| = 3,7 \times 10^{-16}$. Este é o teste mais forte dos três: ele passa apenas se a geometria (as $\mathbf{B}$), o material (a $\mathbf{C}$) e a montagem (as posições de *scatter*) estiverem todos coerentes entre si. Um erro numa única $\mathbf{B}$ já quebraria a rotação.

A Figura 6 mostra o mesmo fato pelo espectro, medido na placa maciça (menor, para caber numa diagonalização densa). Uma $\mathbf{k}_e$ tem posto 6: seis autovalores em torno de 10^{-7} N/m e seis entre 10^8 e 3×10^{10} . A $\mathbf{K}$ inteira (2271×2271) tem seis autovalores em torno de 10^{-4} N/m e o sétimo em $8,2 \times 10^6$: um vão de dez ordens de grandeza que separa os modos rígidos dos modos elásticos.

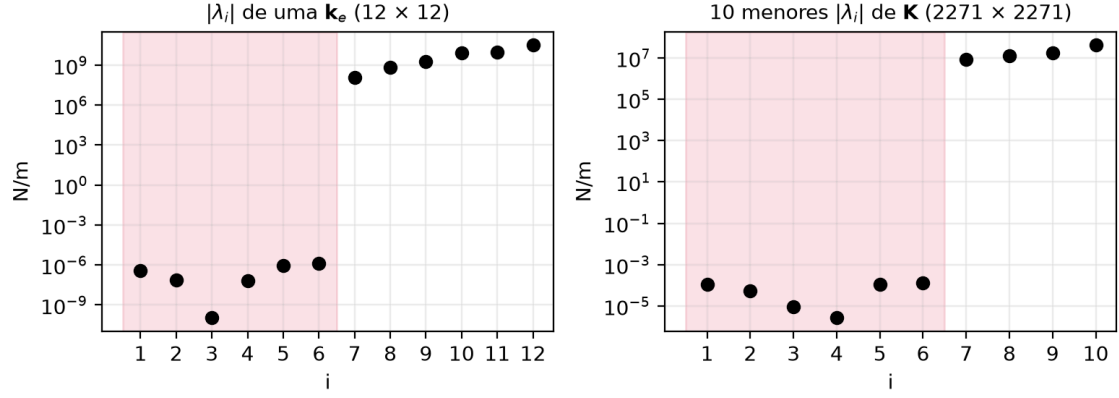


Figura 6: Autovalores em módulo, escala logarítmica. Esquerda: os 12 de uma matriz de elemento; direita: os 10 menores de **K** da placa maciça. A faixa vermelha marca os seis modos de corpo rígido, numericamente zero.

7.3 O que os números da corrida dizem

Quantidade	Placa treliça com furo	Placa maciça com furo
Triângulos na superfície	8192	1456
Nós / tetraedros	4667 / 16 015	757 / 2478
Graus de liberdade n	14 001	2271
Não-zeros de K	488 007 (0,25 %)	78 147 (1,5 %)
Maior entrada de K	$1,4 \times 10^{11}$ N/m	$6,0 \times 10^{11}$ N/m
Assimetria relativa	$2,2 \times 10^{-16}$	$3,0 \times 10^{-16}$
Menor diagonal	$1,1 \times 10^9$ N/m	$1,1 \times 10^{10}$ N/m
$\max \mathbf{K} \mathbf{u}_{\text{rígido}} $, relativo	$3,7 \times 10^{-16}$	$2,9 \times 10^{-16}$
Tempo total	≈ 7 s	≈ 5 s

Tabela 2: Saída das duas configurações do `launch.json`, resolução 10, aço.

A placa maciça tem entradas maiores porque, com a mesma resolução, os seus tetraedros são maiores e mais “gordos” na direção do plano: $\mathbf{k}_e \propto V_e/h^2$ para um elemento de tamanho h , e a treliça é feita de barras finas.

8 O que falta, e por que o script para aqui

Com **K** montada, o caminho até a otimização do artigo é:

1. **Condições de contorno de Dirichlet.** Prescrever $\mathbf{u} = \mathbf{0}$ nos nós de uma face (`solid.constraints[mask, :] = True`). O torch-fem troca as linhas e colunas desses GDL pela identidade, o que remove os seis modos rígidos e torna **K** definida positiva, logo invertível.
2. **Cargas de Neumann.** O vetor $\mathbf{f}$ de Equação 15, distribuído pelos nós (`solid.forces[mask, d] = ...`).
3. **Solução.** $\mathbf{K} \mathbf{u} = \mathbf{f}$ por um solver esparsa direto (`solid.solve(method="spsolve")`). É o gargalo de custo: no artigo, 1,5 dos 1,8 minutos de cada iteração.
4. **Objetivo.** A flexibilidade (*compliance*) $J = \mathbf{f}^T \mathbf{u} = \mathbf{u}^T \mathbf{K} \mathbf{u}$, o dobro da energia de deformação. Minimizá-la é maximizar a rigidez sob a carga dada.
5. **Restrição.** O volume $V = \sum_e V_e$, direto de Equação 11, tem de ficar abaixo de um alvo.

6. **Sensibilidades.** Como J é auto-adjunto, $dJ/dp = -\mathbf{u}^T(\partial \mathbf{K}/\partial p)\mathbf{u}$ para qualquer parâmetro p . A cadeia inteira

$$\hat{\lambda} \rightarrow \lambda(\xi) \rightarrow f_\theta \rightarrow \varphi \rightarrow \text{vértices (FlexiCubes)} \rightarrow \bar{x} \text{ (FFD)} \rightarrow \mathbf{K} \rightarrow J \quad (21)$$

é diferenciável, e o `torch.autograd` a percorre de trás para a frente. É por isso que nada no laço pode sair do torch: um `.numpy()` ou `.detach()` no meio corta a corrente.

7. **Atualização.** O MMA (*Method of Moving Asymptotes*) move os pontos de controle $\hat{\lambda}$ dentro da faixa $[0,15,0,75]$ e o laço recomeça na etapa 3.

`DeepSDFStruct/tests/test_structural_optimization.py` faz tudo isso numa viga em balanço e é o modelo para o próximo script. A única ressalva: ele monta a malha pelo caminho com cavidades descrito na seção 5.3, então os seus números de flexibilidade não correspondem à geometria que a rede descreve.

9 Resumo em uma tabela

Função	Teoria	Equação
<code>plate_with_hole</code>	SDF da célula (rede), empilhamento com $T(x)$, latente constante, furo por diferença booleana, tampas por interseção, spline de deformação.	1, 2, 6, 7
<code>tetrahedral_mesh</code>	Extração do nível zero (FlexiCubes), mapa para metros (FFD), tetraedrização de Delaunay restrita (tetgen), orientação positiva.	9, 11
<code>build_solid</code>	Lei de Hooke isotrópica em Voigt; definição do problema discreto.	12, 14
<code>assemble_stiffness</code>	Matriz de elemento do tetraedro linear e montagem por <i>scatter-add</i> .	17, 18, 19
<code>check_stiffness</code>	Simetria, positividade da diagonal, núcleo de dimensão seis.	20
<code>export</code>	$\mathbf{K}$ em CSR (<code>.npz</code>), malha em <code>.vtk</code> , padrão de esparsidade.	—

Tabela 3: Cada função do script e a teoria que ela implementa.